

Projet Mini-Blog :

Conception et réalisation d'une application web complète

Rapport d'architecture logicielle, choix de conception et implémentation technique

Fiche documentaire

Projet : Mini-Blog

Auteur : Florian Pépin

Version : 1.0

Date : 25 septembre 2026

Table des matières

1	Introduction	2
2	Partie I : Architecture et réalisation du backend	3
2.1	Choix de conception et justifications techniques	3
2.1.1	Axe 1 : Mécanisme d'authentification et gestion de session	3
2.1.2	Axe 2 : Inscription et création du premier administrateur	3
2.1.3	Axe 3 : Règles de publication et modération des commentaires	3
2.1.4	Axe 4 : Cycle de vie et statuts éditoriaux des articles	4
2.1.5	Axe 5 : Déclenchement de la publication et modération	4
2.1.6	Axe 6 : Confidentialité et visibilité des articles	4
2.1.7	Axe 7 : Suppression des données et liens en cascade	5
2.1.8	Axe 8 : Évolution et migrations de la base de données	5
2.1.9	Axe 9 : Conteneurisation et gestion des secrets	5
2.2	Modèle de données et schéma relationnel	6
2.3	Matrice de sécurité et droits d'accès	7
2.4	Validation et tests automatisés du backend	7
3	Partie II : Architecture et réalisation du frontend	8
3.1	Choix de conception du client web	8
3.1.1	Axe 10 : Solution de style et design visuel	8
3.1.2	Axe 11 : Gestion des pannes réseau et arrêt du serveur	8
3.1.3	Axe 12 : Routage et communication avec l'API	8
3.1.4	Axe 13 : Rédaction et mise en page des articles	9
3.1.5	Axe 14 : Sécurisation ergonomique des actions sensibles	9
3.1.6	Axe 15 : Actualisation instantanée et synchronisation réactive des rôles	9
3.2	Écrans et parcours utilisateur	10
4	Partie III : Déploiement, gestion des secrets et conclusion	11
4.1	Orchestration conteneurisée et isolation des secrets	11
4.2	Conclusion et bilan de réalisation	11
A	Annexe 1 : Référentiel des End-points API	12
B	Annexe 2 : Référentiel des tests unitaires et d'intégration	14

1 Introduction

Le projet **Mini-Blog** a pour objectif d'offrir une plateforme moderne, performante et sécurisée de publication et d'interaction communautaire. L'application est architecturée de la façon suivante :

- **Backend RESTful** : Développé en Java 25 avec **Spring Boot 4**, assurant la gestion des utilisateurs, des articles, des commentaires, du hachage BCrypt, des jetons JWT stateless et du contrôle d'accès au niveau ressource.
- **Base de Données Relationnelle** : Propulsée par **PostgreSQL 16**, avec contrôle strict de schéma et migrations SQL versionnées via **Flyway**.
- **Frontend Single Page Application (SPA)** : Développé avec **Angular 18** en architecture *Standalone* intégrale, exploitant les **Signals** réactifs et le design system utilitaire **Tailwind CSS**.
- **Infrastructure & Conteneurisation** : Déploiement automatisé multi-conteneurs via **Docker Compose** avec serveur web **Nginx** servant l'application sur le port conventionnel 4200 et assurant le reverse-proxying interne vers l'API sans friction CORS.

2 Partie I : Architecture et réalisation du backend

2.1 Choix de conception et justifications techniques

2.1.1 Axe 1 : Mécanisme d'authentification et gestion de session

Question : Comment reconnaître un utilisateur connecté et sécuriser ses requêtes ?

Options :

- ☒ Jeton numérique autonome (JWT stateless dans l'en-tête **Authorization**).
- ☐ Jeton stocké dans un cookie de session de navigateur.
- ☐ Session classique où le serveur garde en mémoire tous les utilisateurs connectés.

Pourquoi : Avec une session classique, le serveur doit mémoriser chaque internaute connecté, ce qui consomme de la mémoire et complique les montées en charge. Avec un jeton (JWT), c'est l'inverse : dès que l'utilisateur s'identifie, le serveur lui remet un « badge numérique chiffré » contenant son identifiant et son rôle. À chaque clic ou action, le navigateur présente ce badge. Le serveur n'a qu'à vérifier sa signature sans rien stocker. C'est plus léger, plus rapide, et c'est la façon la plus naturelle de faire communiquer une application Angular avec une API.

2.1.2 Axe 2 : Inscription et création du premier administrateur

Question : Comment permettre aux visiteurs de s'inscrire tout en garantissant qu'un administrateur existe dès le départ ?

Options :

- ☒ Inscription publique pour les utilisateurs réguliers + Création automatique du premier administrateur au démarrage.
- ☐ Inscription publique pour tous, puis promotion manuelle d'un compte en modifiant la base de données.
- ☐ Inscription fermée (seul un administrateur peut créer de nouveaux comptes).

Pourquoi : N'importe quel visiteur peut s'inscrire librement pour commencer à rédiger des articles. En revanche, le rôle d'administrateur donne tous les pouvoirs sur le site et ne doit jamais être accessible lors d'une inscription publique. Pour que la plateforme soit utilisable immédiatement dès le premier lancement sans exiger de manipulation manuelle dans la base de données, le backend crée automatiquement un compte administrateur par défaut s'il n'existe pas encore.

2.1.3 Axe 3 : Règles de publication et modération des commentaires

Question : Qui a le droit de commenter et comment assurer la modération des discussions ?

Options :

- ☒ Commentaires réservés aux utilisateurs connectés sur articles publiés ; modification et suppression par l'auteur ; modération totale par l'administrateur.
- ☐ Commentaires ouverts aux visiteurs anonymes sans compte.
- ☐ Commentaires définitifs (impossibles à modifier une fois postés).

Pourquoi : Ouvrir les commentaires aux anonymes attirerait inévitablement du spam et des messages indésirables : imposer d'être connecté responsabilise les auteurs. De plus, il est illogique de pouvoir commenter un article encore en brouillon : les discussions ne s'ouvrent que sur les articles publics. Enfin, pour le confort d'usage, chaque utilisateur peut corriger ou retirer ses propres messages, tandis que l'administrateur conserve le droit de supprimer n'importe quel commentaire pour faire respecter la charte du blog.

2.1.4 Axe 4 : Cycle de vie et statuts éditoriaux des articles

Question : Comment éviter qu'un administrateur ne publie par erreur un brouillon inachevé ?

Options :

- ☒ Workflow à trois statuts : brouillon privé (DRAFT), en attente de validation (PENDING_REVIEW) et publié (PUBLISHED).
- ☐ Deux statuts simples sans étape explicite de soumission par l'auteur.
- ☐ Publication directe sans modération administrative.

Pourquoi : Si un administrateur pouvait voir et publier n'importe quel brouillon, il risquerait de mettre en ligne le travail non terminé d'un auteur en pleine rédaction. Avec trois statuts, la frontière est claire : tant qu'un article est en brouillon, il est strictement privé à son auteur. Quand l'auteur juge son texte prêt, il le soumet pour validation. L'article est alors gelé pour l'auteur et soumis à l'administrateur, qui peut l'approuver ou le renvoyer en brouillon. Une fois publié, l'article devient public et verrouillé.

2.1.5 Axe 5 : Déclenchement de la publication et modération

Question : Comment concevoir les actions de validation pour éviter les confusions ?

Options :

- ☒ Actions dédiées par rôle : soumission et annulation pour l'auteur, validation, rejet et dépublication pour l'administrateur.
- ☐ Un simple champ de formulaire à cocher lors de l'enregistrement du texte.

Pourquoi : Séparer nettement la modification du contenu des décisions de publication évite toute ambiguïté. L'auteur dispose de ses boutons pour soumettre son texte ou annuler sa soumission s'il souhaite encore le retoucher. L'administrateur dispose de commandes réservées pour valider l'article (mise en ligne), le rejeter (retour en brouillon chez l'auteur) ou le dépublier en cas de nécessité éditoriale.

2.1.6 Axe 6 : Confidentialité et visibilité des articles

Question : Qui doit pouvoir voir les articles selon leur statut ?

Options :

- ☒ Cloisonnement strict : l'auteur ne voit que ses propres brouillons, l'administrateur ne voit que les articles soumis et publiés, et les visiteurs ne voient que les articles publiés.
- ☐ Donner accès à tous les brouillons dans la console d'administration.

Pourquoi : Un brouillon est une ébauche personnelle et confidentielle. L'administrateur n'a pas à surveiller des textes inachevés : son tableau de bord se concentre sur les articles effectivement soumis à modération et les articles en ligne. Les visiteurs, quant à eux, ne voient que les articles officiellement validés.

2.1.7 Axe 7 : Suppression des données et liens en cascade

Question : Que doit-il se passer pour les commentaires lorsqu'un article est supprimé ?

Options :

- ☒ Suppression totale et propre : supprimer l'article efface automatiquement tous ses commentaires.
- ☐ Conserver les commentaires orphelins sans leur article.
- ☐ Masquer l'article sans jamais réellement effacer les données de la base.

Pourquoi : Conserver des commentaires attachés à un article qui n'existe plus encombrerait inutilement la base de données et créerait des incohérences. Supprimer l'article et ses commentaires en un seul geste garantit que la base de données reste parfaitement propre, tout en respectant le droit à l'effacement.

2.1.8 Axe 8 : Évolution et migrations de la base de données

Question : Comment s'assurer que la base de données s'installe à l'identique sur tous les ordinateurs sans risque d'erreur humaine ?

Options :

- ☒ Scripts de création SQL numérotés et automatisés avec Flyway.
- ☐ Laisser le framework Java deviner et modifier les tables tout seul à chaque lancement.
- ☐ Demander à chaque développeur d'exécuter manuellement des commandes dans sa base.

Pourquoi : Laisser un outil automatique modifier la base de données à l'aveugle est une cause majeure de pannes et de pertes de données. Avec Flyway, chaque changement de structure est écrit dans un script SQL clair, daté et vérifié. Au démarrage, l'application lit ces scripts dans l'ordre et met à jour la base sans intervention humaine, assurant une installation infailible sur n'importe quel ordinateur.

2.1.9 Axe 9 : Conteneurisation et gestion des secrets

Question : Comment permettre de démarrer tout le projet en un clic sans risque de fuite de mots de passe ?

Options :

- ☒ Lancement complet via Docker Compose et externalisation des mots de passe dans un fichier de configuration protégé (`.env.example`).
- ☐ Obliger à installer manuellement PostgreSQL, Java et Node.js sur la machine hôte.
- ☐ Écrire les mots de passe et clés secrètes directement dans le code source.

Pourquoi : Docker Compose permet de réunir la base de données, le backend et le frontend dans des boîtes virtuelles isolées (conteneurs) et de tout lancer avec une seule commande (`docker compose up`). Par ailleurs, pour garantir la sécurité, aucun mot de passe n'est écrit « en dur » dans le code : ils sont placés dans un fichier local (`.env`) rigoureusement ignoré par Git, ce qui évite toute fuite accidentelle sur Internet tout en proposant un modèle prêt à l'emploi (`.env.example`).

2.2 Modèle de données et schéma relationnel

Le schéma de la base de données est composé de trois tables principales :

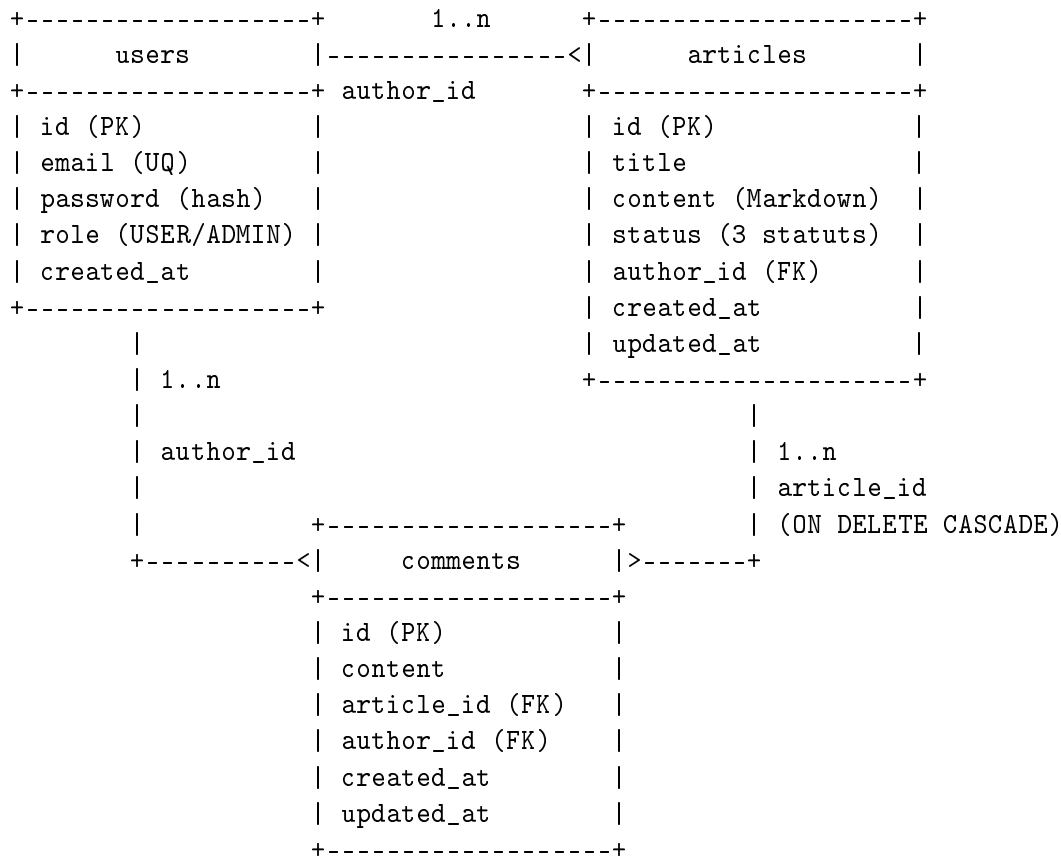


FIGURE 1 – Diagramme relationnel de la base de données

- **users** : Stocke l'adresse email (contrainte d'unicité), le mot de passe haché par l'algorithme BCrypt, et le rôle (**ROLE_USER**, **ROLE_MODERATOR** ou **ROLE_ADMIN**).
- **articles** : Stocke le titre, le contenu rédigé en Markdown, le statut (**DRAFT**, **PENDING_REVIEW** ou **PUBLISHED**), la clé étrangère vers l'auteur (**author_id**), et les timestamps de création et mise à jour.
- **comments** : Stocke le texte du commentaire et les clés étrangères (**article_id**, **author_id**), avec suppression en cascade (**ON DELETE CASCADE**).

2.3 Matrice de sécurité et droits d'accès

La sécurité combine deux niveaux complémentaires : le contrôle d'accès basé sur les rôles (*Role-Based Access Control*) et le contrôle au niveau des instances de ressources (*Resource-Level Security*).

TABLE 1 – Matrice synthétique des droits d'accès par rôle

Opération / res- source	Visiteur	Auteur	Modérateur	Admin
Authentification	Inscription / Login	Session active	Session active	Session active
Articles publiés	Consultation libre	Consultation libre	Consultation libre	Modération & dépublication
Brouillons (DRAFT)	Inaccessibles	Édition & suppression de ses écrits	Édition & suppression de ses écrits	Inaccessibles (privés à l'auteur)
En attente (PENDING)	Inaccessibles	Consultation & annulation de soumission	Consultation & annulation de soumission	Examen, validation & rejet
Commentaires sur publiés	Lecture seule	Ajout, édition & suppression des siens	Ajout, édition des siens & suppression de tous	Modération & suppression de tous
Gestion des utilisateurs	Inaccessible	Inaccessible	Inaccessible	Attribution rôles (Auteur ↔ Modérateur)
Commentaires sur non publiés	× Interdit	× Interdit	× Interdit	× Interdit

2.4 Validation et tests automatisés du backend

Afin de garantir la robustesse des règles de gestion et la stricte étanchéité de la sécurité au niveau ressource, une suite de 23 tests automatisés a été implémentée avec JUnit 5, Mockito et MockMvc.

TABLE 2 – Synthèse concise des 23 tests automatisés du backend

Domaine	Type	Scénarios validés	Tests	Statut
Authentification	Unitaire	Inscription nominale, unicité d'email (409), login valide et génération JWT	3	✓
Articles	Unitaire	Création brouillon forcé, soumission auteur, annulation, rejet admin, validation, dépublication, verrouillages	12	✓
Commentaires	Unitaire	Ajout sur publié, interdiction sur non publié, modification/suppression auteur, modération admin	7	✓
Sécurité globale	Intégration	Scénario complet en 17 étapes (isolation brouillon, soumission, publication admin, cascade)	1	✓
Total (exécutable via mvn test)			23	100%

3 Partie II : Architecture et réalisation du frontend

3.1 Choix de conception du client web

3.1.1 Axe 10 : Solution de style et design visuel

Question : Comment concevoir une interface agréable, moderne et parfaitement lisible sur smartphone comme sur ordinateur ?

Options :

- ☒ Tailwind CSS (Mise en forme utilitaire moderne et sur-mesure).
- ☐ Bootstrap 5 (Composants prédéfinis classiques).
- ☐ Angular Material (Design standardisé Google).
- ☐ CSS pur écrit entièrement à la main sans outil d'aide.

Pourquoi : Plutôt que d'écrire des centaines de lignes de CSS complexes ou de subir des composants rigides difficiles à personnaliser, Tailwind CSS permet de styliser chaque élément directement avec des mots-clés simples et visuels. Cela nous a permis de concevoir des composants sur-mesure très soignés (badges ambre pour les brouillons, badges vert émeraude pour les articles publiés, formulaires épurés), tout en garantissant une application légère et très rapide à charger.

3.1.2 Axe 11 : Gestion des pannes réseau et arrêt du serveur

Question : Que doit voir l'utilisateur si le serveur backend s'arrête ou ne répond plus ?

Options :

- ☒ Approche moderne et discrète : Toast flottant non-bloquant avec compte à rebours de reconnexion et bouton d'action manuelle.
- ☐ Bannière rouge bloquante occupant tout le haut de page.
- ☐ Redirection brutale vers une page d'erreur blanche (503 **Service Unavailable**).

Pourquoi : Si le serveur rencontre une coupure temporaire, rediriger brutalement l'utilisateur vers une page d'erreur ferait perdre tout le texte qu'il était en train d'écrire. À l'inverse, ne rien afficher le laisserait dans l'incompréhension face à des boutons qui ne répondent plus. Nous avons conçu une notification flottante moderne : elle prévient calmement l'utilisateur sans masquer son travail, décompte les secondes avant une nouvelle tentative automatique, et propose un bouton pour réessayer immédiatement dès que la connexion est rétablie.

3.1.3 Axe 12 : Routage et communication avec l'API

Question : Comment faire dialoguer le site web Angular et l'API Spring Boot sans rencontrer de blocages du navigateur ?

Options :

- ☒ Serveur Nginx comme porte d'entrée unique sur le port standard 4200, distribuant le site et relayant les requêtes vers l'API en coulisses.
- ☐ Lancer deux serveurs totalement distincts et configurer des autorisations complexes entre domaines (CORS).

Pourquoi : Par sécurité, les navigateurs internet bloquent fréquemment les échanges lorsqu'un site web tente d'appeler un serveur situé sur une adresse ou un port différent (les fameuses erreurs CORS). Nginx résout ce problème à la racine en agissant comme un « guichet unique » sur le port habituel d'Angular (4200) : pour l'internaute, tout provient du même endroit. Nginx affiche le site web et transmet de manière transparente les demandes de données au serveur Spring Boot en arrière-plan.

3.1.4 Axe 13 : Rédaction et mise en page des articles

Question : Comment permettre aux auteurs d'enrichir le texte de leurs articles sans risquer d'introduire des failles de sécurité ?

Options :

- ☒ Format Markdown léger avec onglet d'aperçu en direct et conversion sécurisée.
- ☐ Texte brut sans aucune mise en forme possible.
- ☐ Éditeur de traitement de texte complexe générant directement du code HTML (WYSIWYG).

Pourquoi : Permettre aux utilisateurs d'écrire du code HTML brut dans un formulaire est extrêmement dangereux : un pirate pourrait y glisser des scripts malveillants pour voler les comptes des lecteurs (attaques XSS). Le format Markdown est la solution idéale : il permet d'ajouter simplement du gras, des titres, des citations et du code informatique avec quelques symboles faciles à retenir (`#`, `**`), tandis que notre application transforme ce texte en un affichage propre, lisible et garanti sans risque de sécurité.

3.1.5 Axe 14 : Sécurisation ergonomique des actions sensibles

Question : Comment prévenir les erreurs de manipulation et suppressions accidentelles sans dégrader l'expérience utilisateur ?

Options :

- ☒ Boîte de dialogue modale réutilisable (`ConfirmDialog`) réactive, avec charte graphique par sévérité (*danger*, *warning*, *success*, *info*) et accessibilité clavier (*Échap*, *Entrée*).
- ☐ Alertes natives du navigateur (`window.confirm()`), austères, bloquantes et non stylisables.
- ☐ Exécution immédiate au premier clic sans aucune confirmation préalable.

Pourquoi : Les suppressions d'articles ou de commentaires ainsi que les changements de statut éditorial (validation et mise en ligne publique, dépublication, soumission pour validation, annulation, renvoi en brouillon) sont des opérations à impact fort ou irréversibles. Un composant modal sur mesure garantit une expérience visuelle moderne, harmonisée avec la charte Tailwind CSS, tout en prévenant les clics involontaires.

3.1.6 Axe 15 : Actualisation instantanée et synchronisation réactive des rôles

Question : Comment garantir qu'un utilisateur promu ou rétrogradé voit son nouveau rôle et ses nouveaux droits immédiatement sans avoir à se reconnecter ?

Options :

- ☒ Interrogation active et transparente du profil connecté via un endpoint dédié (`GET /api/auth/me`) déclenchée au démarrage de l'application et à chaque navigation Angular.

- Maintien des données du profil figées dans le stockage local du navigateur (`localStorage`) jusqu'à une déconnexion et reconnexion manuelle.
- Intégration d'un bus WebSocket persistant, lourd et disproportionné pour le simple rafraîchissement d'habilitations.

Pourquoi : Dans une architecture découplée avec jetons JWT et SPA, les données utilisateur enregistrées dans le navigateur peuvent devenir obsolètes si un administrateur change un rôle côté serveur. En interrogeant légèrement le backend via `GET /api/auth/me` à chaque navigation et rechargement de page, le frontend met à jour instantanément le profil et les permissions en tâche de fond : dès qu'un auteur est promu modérateur, son badge et ses droits de modération apparaissent immédiatement au moindre clic, sans aucune friction.

3.2 Écrans et parcours utilisateur

L'application couvre l'ensemble des parcours fonctionnels d'un blog collaboratif :

- **Visiteur anonyme** : Consulte le flux des articles publiés sur `/articles`, effectue des recherches textuelles, lit les détails d'un article et ses commentaires. Il peut créer un compte sur `/register` ou s'identifier sur `/login`.
- **Auteur authentifié (ROLE_USER)** : Rédige des articles sur `/articles/nouveau` (créés au statut initial `DRAFT`). Il dispose de son espace `/mes-brouillons` pour retrouver ses articles en cours, les modifier, les supprimer ou les soumettre en validation (`PENDING_REVIEW`). Il peut commenter les articles publiés.
- **Modérateur authentifié (ROLE_MODERATOR)** : Auteur disposant de privilèges élargis sur l'espace communautaire. Il conserve toutes les fonctionnalités de rédaction, soumission et gestion de ses propres articles, et bénéficie du droit d'intervention pour supprimer tout commentaire inapproprié sur la plateforme. Son rôle est matérialisé par un badge cyan distinctif dans l'en-tête de navigation.
- **Administrateur (ROLE_ADMIN)** : Dédié exclusivement à l'espace de gestion et de validation `/admin`. Par principe de séparation stricte des responsabilités et respect de l'intégrité éditoriale, l'administrateur n'intervient pas comme lecteur ordinaire, ne rédige pas d'articles et ne retouche pas le texte des auteurs. Son interface et ses droits sont concentrés sur la revue de conformité (validation, rejet, dépublication, suppression d'articles) et la gouvernance des utilisateurs (attribution ou retrait du rôle Modérateur, avec interdiction stricte de promotion au rôle Administrateur et protection du compte admin principal).

Action / Ressource	Visiteur	Auteur	Modérateur	Admin
Consulter les articles publiés	✓	✓	✓	× (Redirection <code>/admin</code>)
Rechercher des articles	✓	✓	✓	×
Commenter un article publié	×	✓	✓	×
Supprimer ses propres commentaires	×	✓	✓	×
Supprimer les commentaires tiers	×	×	✓	✓ (<code>/admin</code>)
Rédiger un article (<code>POST</code>)	×	✓ (<code>DRAFT</code>)	✓ (<code>DRAFT</code>)	×
Modifier le texte (<code>PUT</code>)	×	✓ (Si <code>DRAFT</code>)	✓ (Si <code>DRAFT</code>)	×
Soumettre à validation	×	✓ (Auteur)	✓ (Auteur)	×
Valider et publier	×	×	×	✓ (<code>/admin</code>)
Renvoyer en brouillon	×	×	×	✓ (<code>/admin</code>)
Dépublier un article	×	×	×	✓ (<code>/admin</code>)
Supprimer un article	×	✓ (Ses brouillons)	✓ (Ses brouillons)	✓ (<code>/admin</code>)
Promouvoir/Rétrograder Modérateur	×	×	×	✓ (<code>/admin</code>)
Promouvoir Administrateur	×	×	×	×

TABLE 3 – Matrice de contrôle d'accès basée sur les rôles (RBAC)

4 Partie III : Déploiement, gestion des secrets et conclusion

4.1 Orchestration conteneurisée et isolation des secrets

L'ensemble de la plateforme est orchestré via un fichier `docker-compose.yml` unique pilotant les 3 conteneurs sur un réseau interne :

- **miniblog-postgres** : Base PostgreSQL 16 Alpine avec healthcheck automatisé.
- **miniblog-backend** : API Spring Boot 4 compilée en conteneur multi-stage (Eclipse Temurin 25 JRE Alpine).
- **miniblog-frontend** : Serveur Nginx Alpine servant l'application Angular compilée et assurant le reverse-proxying vers l'API.

Gestion des secrets d'environnement : Les informations sensibles (mots de passe de la base de données, clé secrète HMAC-SHA du JWT, identifiants du compte administrateur) sont gérées selon les meilleures pratiques DevSecOps :

- Un gabarit documenté `.env.example` est versionné dans le dépôt Git comme modèle de référence.
- Un fichier local `.env` contient les secrets actifs chargés par Docker Compose.
- Le fichier `.gitignore` bloque strictement tout envoi de fichiers d'environnement sur le dépôt distant.
- L'application web est exposée sur le port conventionnel Angular 4200 (4200:80).

4.2 Conclusion et bilan de réalisation

La plateforme **Mini-Blog** dispose d'une implémentation fullstack complète, opérationnelle et rigoureusement documentée :

- **Backend** : Architecture Spring Boot 4 en Java 25, persistance PostgreSQL 16 avec migrations Flyway, hachage BCrypt, jetons JWT stateless et sécurité granulaire par ressource.
- **Couverture de tests** : Suite automatisée de 44 tests unitaires et d'intégration validant 100% de la matrice des droits, de la gouvernance des rôles, de la gestion du compte et des règles d'étanchéité éditoriale.
- **Expérience frontend** : Application Angular 18 moderne, performante et réactive exploitant les Signals, un design system moderne avec Tailwind CSS, et une tolérance aux pannes réseau.
- **Déploiement reproductible** : Orchestration conteneurisée via Docker Compose, isolation étanche des secrets d'environnement via `.env.example`, et reverse-proxying Nginx sur le port conventionnel 4200.

A Annexe 1 : Référentiel des End-points API

Le tableau récapitulatif ci-dessous recense l'ensemble des points d'entrée de l'API RESTful exposée par le backend Spring Boot, avec leur méthode HTTP, leur chemin d'accès, les restrictions d'accès et leur rôle fonctionnel :

TABLE 4 – Répertoire complet des endpoints de l'API REST Mini-Blog

Méthode	Chemin (URI)	Accès / Rôle requis	Rôle et description fonctionnelle
1. Authentification (/api/auth)			
POST	/api/auth/register	Public (Anonyme)	Inscription d'un nouvel utilisateur (créé avec le rôle ROLE_USER).
POST	/api/auth/login	Public (Anonyme)	Authentification par email/mot de passe et émission d'un token JWT.
GET	/api/auth/me	Authentifié (Tous)	Récupération du profil synchronisé à jour de l'utilisateur connecté.
2. Gestion du compte personnel (/api/account)			
PATCH	/api/account/email	Authentifié (Auteur / Mod.)	Modification de l'adresse email après vérification du mot de passe actuel.
PATCH	/api/account/password	Authentifié (Auteur / Mod.)	Modification du mot de passe avec validation de l'ancien et confirmation.
DELETE	/api/account	Authentifié (Auteur / Mod.)	Suppression définitive du compte et effacement en cascade (articles et commentaires).
3. Gestion des articles (/api/articles)			
GET	/api/articles	Public (Anonyme)	Liste paginée des articles publiés (ou filtrage avec recherche textuelle).
POST	/api/articles	Authentifié (Auteur / Mod.)	Rédaction d'un nouvel article (créé obligatoirement au statut DRAFT).
GET	/api/articles/my-drafts	Authentifié (Auteur / Mod.)	Consultation privée de l'ensemble de ses brouillons et articles en attente.
GET	/api/articles/{id}	Public / Auteur	Détail d'un article publié (ou d'un brouillon si l'utilisateur en est l'auteur).
PUT	/api/articles/{id}	Auteur (si DRAFT)	Modification du titre et du contenu d'un article en brouillon.
DELETE	/api/articles/{id}	Auteur (brouillon) / Admin	Suppression définitive d'un article et de ses commentaires en cascade.
PATCH	/api/articles/{id}/submit	Auteur propriétaire	Soumission d'un brouillon (DRAFT → PENDING_REVIEW) pour examen.
PATCH	/api/articles/{id}/cancel	Auteur propriétaire	Annulation de la soumission (PENDING_REVIEW → DRAFT) pour modifications.
PATCH	/api/articles/{id}/publish	Administrateur	Validation et publication en ligne d'un article soumis (PUBLISHED).
PATCH	/api/articles/{id}/reject	Administrateur	Rejet d'un article soumis et renvoi à l'auteur en brouillon (DRAFT).
PATCH	/api/articles/{id}/unpublish	Administrateur	Dépublication d'un article public (PUBLISHED → DRAFT).
4. Commentaires (/api/comments)			
GET	/api/articles/{id}/comments	Public (Anonyme)	Liste paginée des commentaires d'un article publié (tri chronologique).
POST	/api/articles/{id}/comments	Authentifié (Auteur / Mod.)	Ajout d'un commentaire sur un article publié.
PUT	/api/comments/{id}	Auteur du commentaire	Modification du texte de son propre commentaire.
DELETE	/api/comments/{id}	Auteur / Modérateur / Admin	Suppression d'un commentaire (auteur du sien, modérateur ou admin de tous).
5. Administration et gouvernance (/api/admin)			
GET	/api/admin/users	Administrateur	Liste complète de tous les utilisateurs inscrits sur la plateforme.
PATCH	/api/admin/users/{id}/role	Administrateur	Attribution ou retrait du rôle Modérateur (ROLE_USER ↔ ROLE_MODERATOR).

B Annexe 2 : Référentiel des tests unitaires et d'intégration

Cette annexe détaille l'intégralité de la suite de tests automatisés mise en place sur le backend Spring Boot (44 tests au total, validant 100% de la matrice de sécurité et des règles métier). Cette suite se décompose en deux catégories complémentaires :

1. **Tests unitaires de services (39 tests)** : Isolant la logique métier pure à l'aide du framework Mockito, sans dépendance à un contexte d'exécution lourd ni à la base de données.
2. **Tests d'intégration de sécurité et de persistance (5 tests)** : Exécutés en contexte complet Spring Boot (@SpringBootTest, MockMvc) avec interactions réelles sur la base PostgreSQL et vérification du cycle de vie des autorisations de bout en bout.

Tableau A : Matrice des Tests Unitaires (Services Métier)

TABLE 5: Référentiel détaillé des tests unitaires de services

Composant	Méthode de test (@Test)	Scénario et comportement attendu
1. Service d'authentification (AuthServiceTest - 5 tests)		
AuthService	register_Success	Inscription valide d'un nouvel auteur : vérification de l'unicité de l'email, encodage BCrypt du mot de passe, persistance en base et génération du jeton JWT.
AuthService	register_EmailAlreadyExists_ThrowsBadRequestException	Rejet immédiat de l'inscription si l'email existe déjà avec levée de <code>BadRequestException</code> et aucune persistance.
AuthService	login_Success	Connexion réussie avec identifiants valides : appel à l' <code>AuthenticationManager</code> et émission du jeton d'authentification JWT.
AuthService	getCurrentUser_Success	Restitution fidèle du profil utilisateur à partir de l'email extrait du jeton JWT dans le contexte de sécurité.
AuthService	getCurrentUser_NotFound_ThrowsException	Détection d'incohérence : levée de <code>BadRequestException</code> si l'utilisateur lié au token est introuvable en base.
2. Service de gestion de compte (AccountServiceTest - 7 tests)		
AccountService	changeEmail_Success	Modification effective de l'adresse email de l'auteur/modérateur après validation réussie de son mot de passe actuel.
AccountService	changeEmail_WrongPassword_ThrowsBadRequest	Rejet de la modification d'email en cas de mot de passe actuel erroné (<code>BadRequestException</code>), interdisant l'usurpation.
AccountService	changeEmail_EmailAlreadyExists_ThrowsBadRequest	Blocage si la nouvelle adresse email est déjà utilisée par un autre compte inscrit sur la plateforme.
AccountService	changePassword_Success	Modification du mot de passe avec validation de l'ancien mot de passe, vérification de la confirmation et réencodage BCrypt.
AccountService	changePassword_MismatchConfirm_ThrowsBadRequest	Rejet immédiat si le nouveau mot de passe et sa confirmation ne sont pas rigoureusement identiques.
AccountService	deleteAccount_Success	Suppression définitive et irréversible du compte utilisateur (Auteur ou Modérateur) après validation du mot de passe.
AccountService	deleteAccount_AdminAccount_ThrowsBadRequest	Protection absolue de l'administration : interdiction formelle pour un compte <code>ROLE_ADMIN</code> de s'auto-supprimer.
3. Service des articles et gouvernance éditoriale (ArticleServiceTest - 13 tests)		
ArticleService	createArticle_AlwaysCreatesDraft	Règle d'or éditoriale : tout nouvel article créé est obligatoirement initialisé au statut <code>DRAFT</code> (brouillon privé).
Suite page suivante...		

Table 5 – suite de la page précédente

Composant	Méthode de test (@Test)	Scénario et comportement attendu
ArticleService	submitArticle_ AuthorCanSubmitDraft_ SetsStatusToPendingReview	L'auteur propriétaire d'un brouillon peut soumettre son article en relecture (DRAFT → PENDING_REVIEW).
ArticleService	submitArticle_ OtherUserCannotSubmitDraft_ ThrowsAccessDenied	Étanchéité des ressources : un utilisateur tiers ne peut pas soumettre le brouillon d'un autre auteur (AccessDeniedException).
ArticleService	cancelSubmission_ AuthorCanCancelPendingArticle_ ReturnsToDraft	Droit de retrait : l'auteur peut annuler sa soumission (PENDING_REVIEW → DRAFT) pour apporter de nouveaux correctifs.
ArticleService	rejectArticle_ AdminRejectsPendingArticle_ ReturnsToDraft	Processus de relecture : l'administrateur rejette un article en attente et le renvoie en brouillon à l'auteur pour révision.
ArticleService	updateArticle_ AuthorCannotUpdate_ PendingArticle_ ThrowsAccessDenied	Gel éditorial : l'auteur ne peut pas modifier un article tant qu'il est en cours d'examen par la modération.
ArticleService	updateArticle_ AuthorCanUpdateDraft	L'auteur propriétaire peut modifier librement le titre et le contenu de son article tant qu'il demeure en statut DRAFT.
ArticleService	updateArticle_ AuthorCannotUpdate_ PublishedArticle_ ThrowsAccessDenied	Immutabilité publique : interdiction formelle pour l'auteur de modifier un article une fois qu'il a été publié en ligne.
ArticleService	updateArticle_ OtherUserCannotUpdateDraft_ ThrowsAccessDenied	Protection d'intégrité : un utilisateur quelconque ne peut altérer le brouillon rédigé par un autre utilisateur.
ArticleService	createArticle_ AdminCannotCreateArticle_ ThrowsAccessDenied	Séparation stricte des rôles : l'administrateur n'a pas le droit de rédiger des articles (AccessDeniedException).
ArticleService	updateArticle_AdminCannotUpdate_ PublishedArticle_ ThrowsAccessDenied	Neutralité éditoriale : l'administrateur n'a pas le droit de modifier le contenu des articles rédigés par les auteurs.
ArticleService	publishArticle_ SetsStatusToPublished	L'administrateur valide l'article soumis et déclenche sa publication publique (PUBLISHED).
ArticleService	unpublishArticle_ SetsStatusToDraft	Action de dépublication : l'administrateur retire un article en ligne et le repasse en brouillon (DRAFT).
4. Service des commentaires (CommentServiceTest - 8 tests)		
CommentService	createComment_ OnPublishedArticle_Success	Tout utilisateur authentifié peut poster un commentaire sous un article ayant le statut PUBLISHED.
CommentService	createComment_OnDraftArticle_ ThrowsBadRequestException	Verrouillage : interdiction formelle de commenter un article qui n'est pas encore publié (BadRequestException).
CommentService	updateComment_AuthorCanUpdate	L'auteur d'un commentaire est autorisé à modifier le texte de son propre commentaire.
CommentService	updateComment_ OtherUserCannotUpdate_ ThrowsAccessDenied	Étanchéité : un utilisateur tiers ne peut pas modifier le commentaire d'un autre membre (AccessDeniedException).
CommentService	deleteComment_AuthorCanDelete	L'auteur d'un commentaire dispose du droit de supprimer son propre commentaire.
CommentService	deleteComment_ AdminCanDeleteAnyComment	Prérogative administrative : l'administrateur peut supprimer n'importe quel commentaire sur l'ensemble du blog.
CommentService	deleteComment_ ModeratorCanDeleteAnyComment	Rôle de modération : le modérateur dispose du droit de supprimer les commentaires inappropriés de tout utilisateur.
CommentService	deleteComment_ OtherUserCannotDelete_ ThrowsAccessDenied	Sécurité : un auteur standard ne peut pas supprimer les commentaires d'autrui (AccessDeniedException).
5. Service d'administration des utilisateurs (UserServiceTest - 6 tests)		
UserService	getAllUsers_AdminSuccess	L'administrateur accède à la consultation exhaustive de tous les utilisateurs enregistrés avec leurs rôles et métadonnées.

Suite page suivante...

Table 5 – suite de la page précédente

Composant	Méthode de test (@Test)	Scénario et comportement attendu
UserService	getAllUsers_UserForbidden	Contrôle d'accès : un utilisateur standard ou modérateur est bloqué en cas de tentative d'accès à la liste des utilisateurs.
UserService	updateUserRole_PromoteToModerator_Success	L'administrateur accorde avec succès le rôle ROLE_MODERATOR à un auteur (ROLE_USER).
UserService	updateUserRole_DemoteToUser_Success	L'administrateur rétrograde un modérateur au rang d'auteur standard (ROLE_USER).
UserService	updateUserRole_PromoteToAdmin_ThrowsBadRequest	Sécurité du système : interdiction absolue de promouvoir un utilisateur au rôle ROLE_ADMIN via l'API.
UserService	updateUserRole_ChangeAdminAccountRole_ThrowsBadRequest	Inviolabilité du compte root : interdiction de rétrograder ou de modifier le rôle du compte administrateur.

Tableau B : Matrice des Tests d'Intégration (Sécurité et Compte)

TABLE 6: Référentiel détaillé des tests d'intégration bout-en-bout

Classe de test	Méthode / Scénario (@Test)	Scénario et comportement attendu
1. Intégration gestion de compte (AccountIntegrationTest - 4 tests)		
AccountIntegrationTest	changeEmail_EndToEnd	Scénario HTTP complet avec MockMvc : mise à jour de l'email en base PostgreSQL, ré-interrogation du profil et vérification de la persistance.
AccountIntegrationTest	changePassword_EndToEnd	Validation bout-en-bout : modification du mot de passe via l'API, tentative de reconnexion réussie avec le nouveau mot de passe et rejet avec l'ancien.
AccountIntegrationTest	deleteAccount_EndToEnd_Cascade	Effacement en cascade : suppression d'un utilisateur possédant des articles et des commentaires, vérification de la purge intégrale en base sans violation de clé étrangère.
AccountIntegrationTest	deleteAccount_AdminBlocked	Tentative de suppression du compte administrateur par appel HTTP : vérification de la réponse 400 Bad Request et intégrité du compte préservée.
2. Intégration cycle de vie et sécurité (SecurityIntegrationTest - 1 test, 17 étapes)		
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 1 : Création DRAFT)	L'Auteur A crée un article : l'API force le statut à DRAFT quel que soit le corps envoyé (201 Created).
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 2 : Liste anonyme)	Un visiteur anonyme interroge GET /api/articles : le brouillon de l'Auteur A est totalement invisible (liste vide).
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 3 : Consultation directe)	Un anonyme tente d'accéder au brouillon via son identifiant : rejet immédiat avec 403 Forbidden.
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 4 : Accès Auteur tiers)	L'Auteur B tente d'accéder au brouillon de l'Auteur A : rejet strict avec 403 Forbidden.
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 5 : Modification propriétaire)	L'Auteur A consulte son propre brouillon (200 OK) et met à jour son titre et son contenu avec succès.
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 6 : Édition frauduleuse)	L'Auteur B tente de modifier le brouillon de l'Auteur A : rejet immédiat avec 403 Forbidden.
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 7 : Soumission frauduleuse)	L'Auteur B tente de soumettre le brouillon de l'Auteur A : rejet immédiat avec 403 Forbidden.
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 8 : Soumission auteur)	L'Auteur A soumet son article en relecture : transition d'état validée de DRAFT vers PENDING_REVIEW .
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 9 : Gel en cours d'examen)	L'Auteur A tente de modifier son article soumis : rejet 403 Forbidden, l'article est gelé durant l'examen.
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 10 : Publication admin)	L'Administrateur valide l'article : transition d'état confirmée de PENDING_REVIEW vers PUBLISHED .
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 11 : Visibilité publique)	L'anonyme peut désormais consulter l'article publié individuellement et le retrouve dans la liste globale.
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 12 : Règle immutabilité)	L'Auteur A tente de modifier son article une fois publié : rejet strict 403 Forbidden (verrou éditorial).
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 13 : Règle suppression)	L'Auteur A tente de supprimer son article publié : rejet strict 403 Forbidden.
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 14 : Commentaire public)	L'Auteur B poste un commentaire sous l'article publié : validation de l'ajout (201 Created).
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 15 : Suppression pirate)	L'Auteur A tente de supprimer le commentaire posté par l'Auteur B : rejet 403 Forbidden.
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 16 : Modération admin)	L'Administrateur supprime le commentaire de B : succès (204 No Content) et purge effective constatée.
SecurityIntegrationTest	fullSecurityLifecycleTest (Étape 17 : Dépublication)	L'Administrateur repasse l'article au statut DRAFT : l'article redevient instantanément invisible pour le public.